Interactive Python Data Structures Guide

An expert guide to core data structures with ADT and direct Python implementations.

Stack (Last-In, First-Out)

A Stack is a linear data structure that follows the **LIFO** principle. The element inserted most recently is the first one to be removed. Operations center around the **TOSS (Top of Stack)**.

Applications

- Function call management (Call Stack)
- Undo/Redo mechanisms in software
- Expression evaluation (Infix to Postfix)
- Backtracking algorithms (e.g., maze solving)

Implementations

1. OOP Class (ADT) - Using TOSS

```
class Stack:
    def __init__(self):
        self._items = []

    def push(self, item):
        self._items.append(item)

    def pop(self):
        if not self.is_empty():
            return self._items.pop()
        raise IndexError("pop from empty stack")

    def peek(self):
        if not self.is_empty():
            return self._items[-1]
        return None

    def is_empty(self):
        return len(self._items) == 0
```

2. Python Internal DS (`list`)

```
# Create a stack using a list
internal_stack = []

# Push items
internal_stack.append("A")
internal_stack.append("B")

# Pop an item
item = internal_stack.pop()
# item is "B"
# internal_stack is now ["A"]
```

Queue (First-In, First-Out)

A Queue is a linear data structure that follows the **FIFO** principle. The element inserted earliest is the first one to be removed. Operations occur at the **front** and **rear** of the queue.

Applications

- CPU and task scheduling
- Printer job spooling
- Breadth-First Search (BFS) algorithm in graphs
- Handling requests on a shared resource

Implementations

1. OOP Class (ADT) - Using Front/Rear

```
from collections import deque

class Queue:
    def __init__(self):
        self._items = deque()

    def enqueue(self, item):
        self._items.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self._items.popleft()
        raise IndexError("dequeue from empty queue")

    def peek(self):
        if not self.is_empty():
            return self._items[0]
        return None

    def is_empty(self):
        return len(self._items) == 0
```

2. Python Internal DS (`collections.deque`)

```
from collections import deque

# Create a queue using deque
internal_queue = deque()

# Enqueue items
internal_queue.append(10)
internal_queue.append(20)

# Dequeue an item
item = internal_queue.popleft()
# item is 10
# internal_queue is now deque([20])
```

Singly Linked List

A Linked List is a linear collection where each element (node) contains data and a pointer to the next node. Unlike arrays, elements are not in contiguous memory. All operations begin from the **root** (the first node).

Applications

- Implementing other data structures like stacks and queues
- Dynamic memory allocation
- Browser history (Doubly Linked Lists)
- Polynomial representation in algebra

Implementations

1. OOP Class (ADT) - Using Root

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.root = None

    def insert_at_start(self, data):
        new_node = Node(data)
        new_node.next = self.root
        self.root = new_node

    def display(self):
        current = self.root
        elements = []
        while current:
            elements.append(str(current.data))
            current = current.next
        print(" -> ".join(elements))
```

2. Conceptual Internal DS (`list`)

```
# This is NOT a true linked list
# but conceptually represents the
# structure of data and pointers.
conceptual_ll = [
    {'data': 'A', 'next': 1},
    {'data': 'B', 'next': 2},
    {'data': 'C', 'next': -1}
]

# Access root data
print(conceptual_ll[0]['data']) # 'A'
```

Tree (Binary Search Tree)

A Tree is a non-linear, hierarchical structure. A **Binary Search Tree (BST)** is a common variant where for any given node (starting from the **root**), all values in its left subtree are smaller, and all values in its right subtree are larger.

Applications

- Database indexing for efficient searching
- File systems and directory structures
- Syntax analysis in compilers (Parse Trees)
- Organizing data for quick lookup

Implementations

1. OOP Class (ADT) - Using Root

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert_bst(root, data):
    if root is None:
        return TreeNode(data)
    if data < root.data:
        root.left = insert_bst(root.left, data)
    elif data > root.data:
        root.right = insert_bst(root.right, data)
    return root

def inorder_traversal(root):
    if root:
        inorder_traversal(root.left)
        print(root.data, end=" ")
        inorder_traversal(root.right)
```

2. Conceptual Internal DS (`dict`)

```
# Conceptual tree using nested dicts
conceptual_tree = {
    'data': 50,
    'left': {
        'data': 30,
        'left': None,
        'right': None
    },
    'right': {
        'data': 70,
        'left': None,
        'right': None
    }
}
# Access root data
print(conceptual_tree['data']) # 50
```

Graph

A Graph is a non-linear structure of vertices (nodes) and edges. This ADT uses an **Adjacency Matrix**, a $V \times V$ grid where an entry at `(i, j)` indicates an edge between vertex `i` and vertex `j`.

Applications

- Social networks (e.g., Facebook, LinkedIn)
- Mapping and navigation (e.g., Google Maps)
- Web page crawling and network modeling
- Task scheduling and dependency management

Implementations

1. OOP Class (ADT) - Adjacency Matrix

```
class Graph:
    def __init__(self, num_vertices):
        self.num_vertices = num_vertices
        self.adj_matrix = [[0] * num_vertices
                           for _ in range(num_vertices)]

    def add_edge(self, u, v, weight=1):
        if u < self.num_vertices and v < self.num_vertices:
            # For undirected graph
            self.adj_matrix[u][v] = weight
            self.adj_matrix[v][u] = weight
        else:
            raise IndexError("Vertex index out of bounds.")

    def display(self):
        for row in self.adj_matrix:
            print(row)
```

2. Python Internal DS (`dict`) - Adjacency List

```
# Adjacency List using a dictionary
# More flexible for non-integer vertices
internal_graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D'],
    'C': ['A', 'D'],
    'D': ['B', 'C', 'E'],
    'E': ['D']
}

# Get neighbors of 'D'
print(internal_graph['D'])
# Output: ['B', 'C', 'E']
```


Made for Career Credentials

Curated and structured by Amar Panchal, Career Mentor and Co-founder for Career Credentials.

Connect with Amar Panchal on:

LinkedIn | Instagram | Facebook | YouTube | Website